0332. Reconstruct Itinerary

| Category: Advanced Graphs | LeetCode: https://leetcode.com/problems/recons

Problem

- Given airline tickets [from, to], build the itinerary starting at "JFK" that uses every ticket exactly once. If several exist, return the lexicographically smallest.

Intuition

- Using every edge exactly once is an Eulerian path.
- Hierholzer's algorithm: DFS greedily taking the smallest destination; when a node has no unused edges left, prepend it to the route. Dead ends are naturally pushed to the end.

Brute Force

- Backtracking over sorted destinations, undoing on dead ends: correct but exponential in the worst case.

Optimal Approach

- Map airport → min-heap of destinations.
- Dfs(a): while heap(a) non-empty, dfs(heap.poll()). Then addFirst(a).
- Result is the route list.

Worked Example

```
[[JFK,KUL],[JFK,NRT],[NRT,JFK]].

dfs JFK -> KUL (smallest) : KUL has no edges -> route [KUL]
back at JFK -> NRT -> JFK : JFK empty now -> route [JFK,KUL]
NRT done -> [NRT,JFK,KUL] ; JFK done -> [JFK,NRT,JFK,KUL]
```

Java Solution

```java
public class ReconstructItinerary {
    private final Map<String, PriorityQueue<String>> g = new HashMap<>();
    private final LinkedList<String> route = new LinkedList<>();

    public List<String> findItinerary(List<List<String>> tickets) {
        for (List<String> t : tickets) g.computeIfAbsent(t.get(0), k -> new PriorityQueue<>()).add(t.get(1));
        dfs("JFK");
        return route;
    }

    private void dfs(String a) {
        PriorityQueue<String> pq = g.get(a);
        while (pq != null && !pq.isEmpty()) dfs(pq.poll());
        route.addFirst(a);
    }
}
```

Complexity

- Time: O(E log E). Space: O(E).

Pitfalls

- Greedy "smallest first" without post-order insertion fails on the KUL dead-end example.